Введение в JavaScript. Переменные и типы данных

Лекция №1

Немного о себе

- Работаю в компании DPD
- Frontend разработчик
- Участвую в разработке крупных проектов на JavaScript (TypeScript)
- Опыт в коммерческой разработке более 3 лет
- JavaScript, TypeScript, React, Vue, NodeJS
- Проводил стажировку для студентов в компании LAD

Ход занятия

- Введение в JavaScript
- Переменные
- Типы данных

Что такое JavaScript?

JavaScript - язык программирования с динамической типизацией, который позволяет Вам создать динамически обновляемый контент, управляет мультимедиа, анимирует изображения, впрочем, делает всё, что угодно.

Ну или все, что угодно, удивительно, чего можно достичь с помощью нескольких строк JavaScript кода. Данная презентация так же была создана с помощью ПО, написанном на JavaScript.

Преимущества JavaScript?

- Язык браузеров. Включён по умолчанию
- Широкая распространенность
- Полная интеграция с HTML/CSS
- Язык высокого уровня
- Быстрый для пользователя
- Широкая область использования
- Наличие “языков-надстроек” над JavaScript
- Невысокий порог вхождения
- Активное развитие языка
- Большое комьюнити

JavaScript и область применения

HTML и CSS

HTML - это язык разметки, который мы используем для визуального и смыслового структурирования нашего web контента, например, определяем параграфы, заголовки, таблицы данных, или вставляем изображения и видео на страницу.

CSS - это язык стилей с помощью которого мы придаем стиль отображения нашего HTML контента, например придаем цвет фону (background) или шрифту.

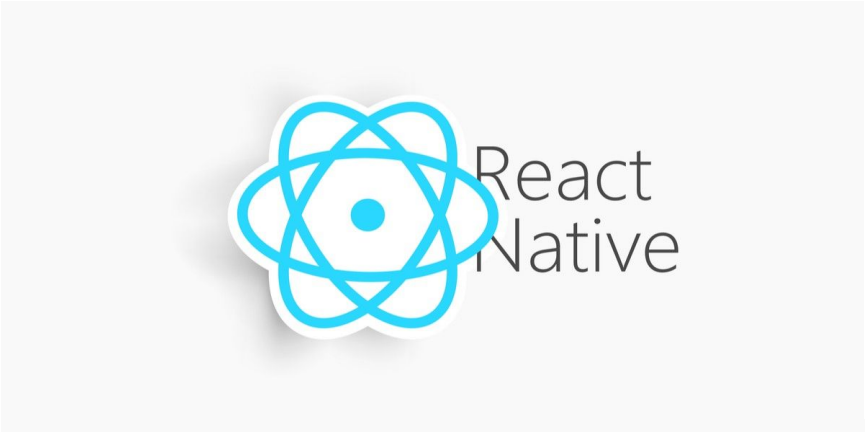
Использование JavaScript?

- **Frontend =>** Реализация клиентских приложений. Реализация визуальной части.
- **Backend =>** Реализация серверных приложений. Реализация бизнес-логики приложений.
- **Mobile =>** Реализация приложений для мобильных устройств.
- **Desktop =>** Реализация приложений для настольных ПК.

Инструменты Frontend

Инструменты Backend

Инструменты Mobile



METEOR

Инструменты Desktop

Недостатки JavaScript

- В JavaScript нет компилятора
- Широкая распространенность
- Непривычная объектная модель
- Однопоточность

Топ языков программирования

Место	Язык программирования	Основные сферы применения	Ключевые характеристики	Популярность (примерные позиции в рейтингах)
1	**Python**	Data Science, AI/ML, веб-бэкенд (Django, Flask), автоматизация, научные вычисления	Простой синтаксис, богатые библиотеки, высокий спрос на рынке	TIOBE: 1, PYPL: 1, Stack Overflow: 1
2	**JavaScript**	Веб-разработка (фронтенд и бэкенд с Node.js), мобильные приложения (React Native)	Единственный язык для браузерного фронтенда, огромная экосистема (npm)	TIOBE: 3, PYPL: 2, Stack Overflow: 1 (часто паритет с Python)
3	**Java**	Корпоративные приложения, Android-разработка, большие данные (Hadoop)	Кроссплатформенность (JVM), надежность, сильная типизация	TIOBE: 4, PYPL: 3, Stack Overflow: 4
4	**C#**	Разработка игр (Unity), корпоративный софт под Windows, веб-сервисы (ASP.NET)	Мощный и современный язык от Microsoft, интеграция с экосистемой .NET	TIOBE: 5, PYPL: 4, Stack Overflow: 6
5	**C/C++**	Системное программирование, игры, высокопроизводительные вычисления, встраиваемые системы	Высокая производительность, низкоуровневый контроль памяти	TIOBE: 2 (C) и 6 (C++), PYPL: 5 (вместе), Stack Overflow: 10 (C++)
6	**PHP**	Веб-бэкенд (около 75% всех сайтов используют PHP, включая WordPress)	Широкая распространенность в вебе, простота для начинающих	TIOBE: 7, PYPL: 7, Stack Overflow: 10+
7	**TypeScript**	Крупная веб-разработка (фронтенд и бэкенд с Node.js)	Статическая типизация поверх JavaScript, повышает надежность кода	Стремительно растет. PYPL: 6, любимый язык в опросе Stack Overflow
8	**Go (Golang)**	Бэкенд-сервисы, облачные приложения, CLI-утилиты (от Google)	Простота, высокая производительность, отличная поддержка многопоточности	TIOBE: 10, PYPL: 11, высокий спрос на рынке
9	**Kotlin**	Android-разработка (официально поддерживается Google), бэкенд	Современная и лаконичная альтернатива Java, полная совместимость с ней	Любимый язык после Rust (Stack Overflow), основной для Android
10	**Rust**	Системное программирование, где критична безопасность и производительность (браузеры, ОС)	Высокая скорость и гарантия безопасности памяти без сборщика мусора	**Самый любимый язык 8 лет подряд (Stack Overflow)**

Стандарты языка
Среда исполнения

Что есть JavaScript?

ECMAScript — спецификация скриптового языка программирования

ES5, ES6 … ES14

JavaScript — язык программирования, одна из реализаций спецификации ECMAScript (наряду с JScript и ActionScript), их ещё называют диалектами ECMAScript

ECMA-262 — стандарт компании Ecma International, по которому разрабатывается спецификация ECMAScript

Последняя стабильная версия: ECMAScript 2023 (ES14), которая была официально выпущена в июне 2023 года.

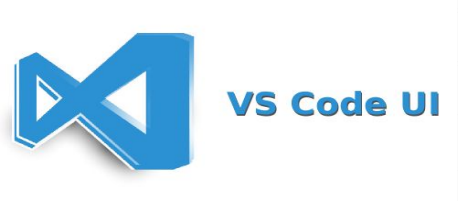
Как запустить JavaScript?

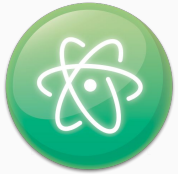
- **Браузер**: через консоль разработчика или онлайн редактор кода
- **NodeJS**: через консоль терминала или через редактор кода

Редакторы кода

Онлайн редакторы кода JavaScript

- JS Bin
- JSFiddle
- CodePen

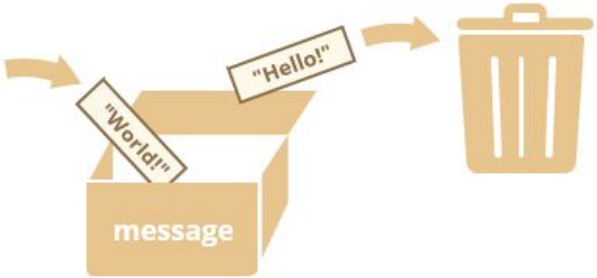




Переменные

Переменные

- Переменные - это «именованное хранилище» для данных (в памяти).
- При этом значение переменных может изменяться по ходу выполнения скрипта.



Имена переменных в JavaScript

- Имя переменной должно содержать только буквы, цифры или символы $ и _
- Первый символ не должен быть цифрой

userName	7a
test123	user-name
$ _	#%()+*

Влияние регистра

Переменные с именами **apple** и **APPLE** – это две разные переменные.

Нелатинские буквы разрешены, но не рекомендуются

let имя = '...';
let 我 = '...';

* Очень частая ошибка - русская буква 'с' в имени переменной (спасает редактор кода)

Зарезервированные имена

- break
- case
- class
- catch
- const
- continue
- debugger
- default
- delete
- do
- else
- export
- extends
- finally
- for
- function
- if
- import
- in
- instanceof
- let
- new
- return
- super
- switch
- this
- throw
- try
- typeof
- var
- void
- while
- with
- yield

Нотации именования

Нотации именования переменных (виды наименований):

- camelCase

- PascalCase
- snake_case
- kebab-case

Пример: camelCase: let myVeryLongName;

Средняя длина имени переменной от 1 до 4 слов.

Правильные имена переменных

Корректно: userName , paymentType

Некорректно: a, b, c, value, _, $

Условия именования:

Предикаты - переменная проверки, проверяет (да) или (нет): isAdmin

Вхождение - имеет что то (да) или (нет): hasChildren

Количество - какое то количество: peopleCount

Правильные имена переменных

Создания переменной в JavaScript

Для создания переменной в JavaScript используйте ключевые слова:

- **let** - это обычная переменная которую, можно изменить;

  let userName = “Alex”;

- **const** - это константа, их нельзя изменить. Попытка сделать это приведёт к ошибке;

  const age = 18;

  const COLOR_BLACK = '#000';

- **var**

Что нужно знать про var

- Устаревший способ
- Из-за функциональной области видимости, способствует странному поведению и не очевидным багам если не знать как с ней работать (глобальные переменные).
- Не используется в новых проектах
- С большей долей вероятности вы ее встретите если пойдете работать в компанию где нужно поддерживать старый проект.
- С большей долей вероятности вы его встретите в коде чужих библиотек
- С большей долей вероятности вас спросят про него на собеседовании

Хорошая практика

- Всегда использовать **const** если переменная не изменяется
- Не стоит повторно использовать переменные. лучше создать новые
- Избегайте использования var

Практика

Придумать наименования переменных для следующих примеров:

- Переменная для “названия нашей планеты”?
- Переменная для “текущее время пользователя”?
- Переменная которая показывает “количества статей”?
- Переменная которая показывает “это оплата наличными деньгами или нет”?
- Три переменные для хранения Ф.И.О

Типы данных

Типы данных

JavaScript - имеет динамическую типизацию. Переменная в JavaScript может содержать любые данные. В один момент там может быть строка, а в другой – число.

Есть восемь основных типов данных в JavaScript:

- number
- string
- boolean
- null
- undefined
- object
- symbol
- bigint

Числа - number

- **number** – для любых чисел: целочисленных или чисел с плавающей точкой.

```
const age = 18;
const pi = 3.14;
```

- **Infinity** - математическая бесконечность (спец-е числовые значения)

```
alert( 1 / 0 );
```

- **NaN** - означает вычислительную ошибку (спец-е числовые значения)

```
alert( 'Alex' / 0 );
```

Строки - string

string – в JavaScript должна быть заключена в кавычки.

- Двойные кавычки:

  let userName = "Alex";

- Одинарные кавычки:

  let userName = 'Alex';

- Обратные кавычки:

  let userName = 'Alex';

  let helloUser = `Hello ${userName}`; //интерполяция выражений

Булевый - boolean

boolean – может принимать только два значения: true (истина) и false (ложь).

let isAdmin = false;

let isOpen = true;

Значение null

null – отдельный тип, специальное значение, которое представляет собой «ничего», «пусто».

```
let userName = null;
```

Значение undefined

undefined – отдельный тип, означает, что «значение не было присвоено», значение неизвестно, ничего не вернулось и не получили

```
let userName;

alert(userName); // выведет "undefined"
```

Объекты - object

object – все остальные типы выше называются «примитивными». Объекты же используются для хранения коллекций данных или более сложных объектов.

```
let user = {
    name: "Alex",
    age: 25
};
```

Представляет собой коллекцию свойств (переменных), где переменные доступны по ключу имени в объекте user.name.

Символы - symbol

symbol – так же примитивный тип как и number или string. Используется для создания уникальных идентификаторов объектов.

Грубо говоря его можно задать как скрытое свойство объекта, которое не будет видно в стандартных функциях для работы с объектами и циклах перебора свойств объекта. Редко используются.

Тип - bigint

bigint – Тип BigInt был добавлен в JavaScript, чтобы дать возможность работать с целыми числами произвольной длины.

Тип «number» не может содержать числа больше, чем 2^{53} (или меньше, чем -2^{53} для отрицательных). Это техническое ограничение вызвано их внутренним представлением. Используется если нужны действительно гигантские числа, например в криптографии или при использовании метки времени («timestamp») с микросекундами.

Оператор typeof

Оператор typeof возвращает тип аргумента. Это полезно, когда мы хотим обрабатывать значения различных типов по-разному или просто хотим сделать проверку.

У него есть два синтаксиса:

- Синтаксис оператора: typeof x
- Синтаксис функции: typeof(x)

Исключения

- Результатом вызова typeof null является "object". Это неверно и это официально признанная ошибка в языке. Ее не могут поправить из-за совместимости, т.к уже очень много кода написано где это используется.
- Вызов typeof alert возвращает "function", потому что alert является функцией. Но в JavaScript нет специального типа «функция». Функции относятся к объектному типу.